```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import urllib.request
import time

# ----------------------------------------------------------
# INPUT IMAGE
# ----------------------------------------------------------
url = "https://raw.githubusercontent.com/opencv/opencv/master/samples/data/butterfly.jpg"
urllib.request.urlretrieve(url, "input.jpg")

img = cv2.imread("input.jpg")

if img is None:
    raise Exception("Image could not be loaded")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# ----------------------------------------------------------
# FEATURE DETECTION USING SIFT
# ----------------------------------------------------------
sift = cv2.SIFT_create()

thresholds = [50, 100, 150, 200, 250]
results = []

for threshold in thresholds:

    start = time.perf_counter()

    # Detect features
    keypoints, descriptors = sift.detectAndCompute(gray, None)

    # Filter keypoints based on response
    filtered = [kp for kp in keypoints if kp.response >= threshold]

    end = time.perf_counter()

    count = len(filtered)

    h, w = gray.shape
    density = count / (h * w) * 100000

    execution_time = (end - start) * 1000

    results.append((threshold, count, density, execution_time))

# ----------------------------------------------------------
# PRINT RESULTS
# ----------------------------------------------------------
print("\nFEATURE DETECTION DENSITY ANALYSIS")
print("-" * 60)
print("Threshold\tFeatures\tDensity/100k pixels\tTime(ms)")

for r in results:
    print(f"{r[0]}\t\t{r[1]}\t\t{r[2]:.2f}\t\t\t{r[3]:.3f}")

# ----------------------------------------------------------
# VISUALIZE FEATURES
# ----------------------------------------------------------
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

for i, threshold in enumerate([50, 150, 250]):

    keypoints, _ = sift.detectAndCompute(gray, None)
    filtered = [kp for kp in keypoints if kp.response >= threshold]

    output = cv2.drawKeypoints(
        img,
        filtered,
        None,
        flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
    )

    axes[i].imshow(cv2.cvtColor(output, cv2.COLOR_BGR2RGB))
    axes[i].set_title(f"Threshold = {threshold}\nFeatures = {len(filtered)}")
    axes[i].axis("off")

plt.tight_layout()
plt.savefig("Q26_feature_density.png")
plt.show()

# ----------------------------------------------------------
# GRAPH
# ----------------------------------------------------------
counts = [r[1] for r in results]
times = [r[3] for r in results]

plt.figure(figsize=(8, 5))
plt.plot(thresholds, counts, marker="o")
plt.xlabel("Feature Response Threshold")
plt.ylabel("Number of Features")
plt.title("Feature Density vs Detection Threshold")
plt.grid(True)
plt.savefig("Q26_density_graph.png")
plt.show()
```

```
plt.figure(figsize=(8, 5))
plt.plot(counts, times, marker="o")
plt.xlabel("Number of Detected Features")
plt.ylabel("Execution Time (ms)")
plt.title("Computational Time vs Feature Count")
plt.grid(True)
plt.savefig("Q26_complexity_graph.png")
plt.show()
```

```
FEATURE DETECTION DENSITY ANALYSIS
------------------------------------------------------------
Threshold   Features    Density/100k pixels  Time(ms)
50      0       0.00            118.860
100     0       0.00            77.070
150     0       0.00            75.805
200     0       0.00            75.646
250     0       0.00            74.483
```